

Assignment

Stored Procedures & UDFs

Hands-on Scenario Based Practice

Q1. Scalar Function — Grade to Points

Create a function academics.grade_to_points(p_grade CHAR(2)) that returns NUMERIC points: A+=10, A=9, B+=8, B=7, C=6, F=0, anything else NULL. Test it with SELECT academics.grade_to_points('A+');

```
234 CREATE OR REPLACE FUNCTION academics.grade_to_points(  
235     p_grade CHAR(2)  
236 )  
237 RETURNS NUMERIC  
238 LANGUAGE plpgsql  
239 AS $$  
240 BEGIN  
241     IF p_grade = 'A+' THEN  
242         RETURN 10;  
243     ELSIF p_grade = 'A' THEN  
244         RETURN 9;  
245     ELSIF p_grade = 'B+' THEN  
246         RETURN 8;  
247     ELSIF p_grade = 'B' THEN  
248         RETURN 7;  
249     ELSIF p_grade = 'C' THEN  
250         RETURN 6;  
251     ELSIF p_grade = 'F' THEN  
252         RETURN 0;  
253     ELSE  
254         RETURN NULL;  
255     END IF;  
256 END;  
257 $$;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 89 msec.

TEST:

```
260 SELECT academics.grade_to_points('A+');  
261
```

Data Output Messages Notifications

SQL

grade_to_points	
numeric	
1	10

Q2. Function with Default Parameter

Create academics.cgpa_category(p_cgpa NUMERIC, p_pass_mark NUMERIC DEFAULT 6.0) that returns TEXT: 'Excellent' (>=9), 'Good' (>=7), 'Average' (>= pass_mark), 'Fail' (below pass_mark), 'Unknown' (NULL cgpa). Test with and without the second argument.

```
264 CREATE OR REPLACE FUNCTION academics.cgpa_category(  
265     p_cgpa NUMERIC,  
266     p_pass_mark NUMERIC DEFAULT 6.0  
267 )  
268 RETURNS TEXT  
269 LANGUAGE plpgsql  
270 AS $$  
271 BEGIN  
272     IF p_cgpa IS NULL THEN  
273         RETURN 'Unknown';  
274     ELSIF p_cgpa >= 9 THEN  
275         RETURN 'Excellent';  
276     ELSIF p_cgpa >= 7 THEN  
277         RETURN 'Good';  
278     ELSIF p_cgpa >= p_pass_mark THEN  
279         RETURN 'Average';  
280     ELSE  
281         RETURN 'Fail';  
282     END IF;  
283 END;  
284 $$;  
285
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 100 msec.

Test without second arguments:

```
287 SELECT academics.cgpa_category(9.2);  
288 SELECT academics.cgpa_category(7.5);  
289 SELECT academics.cgpa_category(6.5);  
290 SELECT academics.cgpa_category(5.5);  
291 SELECT academics.cgpa_category(NULL);  
292
```

Data Output Messages Notifications

	cgpa_category text
1	Unknown

Test with second argument

```
293 SELECT academics.cgpa_category(5.5, 5.0);
```

Data Output Messages Notifications

	cgpa_category text
1	Average

Q3. Set-Returning Function

Create `academics.get_dept_students(p_dept_id INTEGER)` that RETURNS TABLE(`student_id` INTEGER, `full_name` VARCHAR, `cgpa` NUMERIC). It should return all students of that department ordered by `cgpa` DESC. Test with `SELECT * FROM academics.get_dept_students(1);`

```
296 CREATE OR REPLACE FUNCTION academics.get_dept_students(  
297     p_dept_id INTEGER  
298 )  
299 RETURNS TABLE(  
300     student_id INTEGER,  
301     full_name VARCHAR,  
302     cgpa NUMERIC  
303 )  
304 LANGUAGE plpgsql  
305 AS $$  
306 BEGIN  
307     RETURN QUERY  
308     SELECT  
309         s.student_id,  
310         s.student_name,  
311         s.cgpa  
312     FROM academics.students s  
313     WHERE s.dept_id = p_dept_id  
314     ORDER BY s.cgpa DESC;  
315 END;  
316 $$;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 65 msec.

Test :

```
319 SELECT *  
320 FROM academics.get_dept_students(1);
```

Data Output Messages Notifications

	student_id integer	full_name character varying	cgpa numeric
1	9999	Test Student	[null]
2	3	Priya Verma	9.12
3	2	Reetiks	8.65
4	1	Sunakshi	8.50
5	4	Vanshika	7.89

Q4. Function — Calculate Student GPA

Create `academics.calc_gpa(p_student_id INTEGER)` that computes the average grade points of all graded enrollments for that student (use your `grade_to_points` function). Return NULL if the student has no graded enrollments. Test for at least two students.

```
324 CREATE OR REPLACE FUNCTION academics.calc_gpa(  
325     p_student_id INTEGER  
326 )  
327 RETURNS NUMERIC  
328 LANGUAGE plpgsql  
329 AS $$  
330 DECLARE  
331     v_gpa NUMERIC;  
332 BEGIN  
333     SELECT AVG(academics.grade_to_points(e.grade))  
334     INTO v_gpa  
335     FROM academics.enrollments e  
336     WHERE e.student_id = p_student_id  
337           AND e.grade IS NOT NULL;  
338  
339     RETURN v_gpa;  
340 END;  
341 $$;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 89 msec.

Test for student1

```
344 SELECT academics.calc_gpa(1);
```

Data Output Messages Notifications		
	calc_gpa numeric	
1	8.666666666666667	

Test for another student

346

SELECT

academics.calc_gpa(2);

Data Output

Messages

Notifications

calc_gpa

numeric

1

8.666666666666667

Test multiple students together

344

345

346

347

348

349

350

351

SELECT academics.calc_gpa(1);

SELECT academics.calc_gpa(2);

SELECT

student_id,

academics.calc_gpa(student_id) AS gpa

FROM academics.students;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	student_id [PK] integer	gpa numeric
1	3	8.000000000000000
2	2	8.666666666666667
3	4	9.000000000000000
4	9999	[null]
5	2001	[null]
6	1	8.666666666666667

Q5. Basic Procedure — Enroll Student

Create a procedure `academics.enroll_student(p_student_id INTEGER, p_course_id INTEGER, p_grade CHAR(2) DEFAULT NULL)` that inserts a row into `enrollments`. Test it with `CALL`.

```
354
355 CREATE OR REPLACE PROCEDURE academics.enroll_student(
356     p_student_id INTEGER,
357     p_course_id INTEGER,
358     p_grade CHAR(2) DEFAULT NULL
359 )
360 LANGUAGE plpgsql
361 AS $$
362 BEGIN
363     INSERT INTO academics.enrollments(
364         student_id,
365         course_id,
366         grade
367     )
368     VALUES (
369         p_student_id,
370         p_course_id,
371         p_grade
372     );
373 END;
374 $$;
```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 183 msec.

Test

```
376 CALL academics.enroll_student(1, 3, 'A');
377
```

Data Output Messages Notifications

ERROR: duplicate key value violates unique constraint "enrollments_pkey"
Key (student_id, course_id)=(1, 3) already exists.

SQL state: 23505

Detail: Key (student_id, course_id)=(1, 3) already exists.

Context: SQL statement "INSERT INTO academics.enrollments(
 student_id,
 course_id,
 grade
)
VALUES (
 p_student_id,
 p_course_id,
 p_grade
)"

PL/pgSQL function academics.enroll_student(integer,integer,character) line 3 at SQL statement

Check

```
378 SELECT *
379 FROM academics.enrollments
380 WHERE student_id = 1
381 AND course_id = 3;
```

Data Output Messages Notifications

	student_id [PK] integer	course_id [PK] integer	enroll_date date	grade character (2)
1	1	3	2026-08-02	B+

Q6. Procedure with Validation EXCEPTION

Enhance (or recreate) the enroll procedure so that:

- It raises an EXCEPTION if the student_id does not exist
- It raises an EXCEPTION if the course_id does not exist
- It catches unique_violation and raises a NOTICE that the student is already enrolled

```
383
384 CREATE OR REPLACE PROCEDURE academics.enroll_student(
385     p_student_id INTEGER,
386     p_course_id INTEGER,
387     p_grade CHAR(2) DEFAULT NULL
388 )
389 LANGUAGE plpgsql
390 AS $$
391 BEGIN
392     IF NOT EXISTS (
393         SELECT 1
394         FROM academics.students
395         WHERE student_id = p_student_id
396     ) THEN
397         RAISE EXCEPTION 'Student ID % does not exist', p_student_id;
398     END IF;
399
400     IF NOT EXISTS (
401         SELECT 1
402         FROM academics.courses
403         WHERE course_id = p_course_id
404     ) THEN
405         RAISE EXCEPTION 'Course ID % does not exist', p_course_id;
406     END IF;
407
408     BEGIN
409         INSERT INTO academics.enrollments(
410             student_id,
411             course_id,
412             grade
413         )
414         VALUES (
415             p_student_id,
416             p_course_id,
417             p_grade
418         );
419     EXCEPTION
420     WHEN unique_violation THEN
421         RAISE NOTICE 'Student % is already enrolled in course %',
422             p_student_id,
423             p_course_id;
424     END;
425
426 END;
427
428 $$;
```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 91 msec.

Test successful enrolment

433	
434	CALL academics.enroll_student(1, 4, 'B+');
435	
436	
437	
Data Output Messages Notifications	
NOTICE: Student 1 is already enrolled in course 4 CALL	
Query returned successfully in 71 msec.	

Test duplicate enrolment

436	CALL academics.enroll_student(1, 4, 'B+');
437	
Data Output Messages Notifications	
NOTICE: Student 1 is already enrolled in course 4 CALL	
Query returned successfully in 117 msec.	

Test invalid student

438	
439	CALL academics.enroll_student(9999, 1, 'A');
Data Output Messages Notifications	
ERROR: Course ID 1 does not exist CONTEXT: PL/pgSQL function academics.enroll_student(integer,integer,character) line 17 at RAISE SQL state: P0001	

Test invalid course

442	CALL academics.enroll_student(1, 9999, 'A');
443	
444	
445	
446	
Data Output Messages Notifications	
ERROR: Course ID 9999 does not exist CONTEXT: PL/pgSQL function academics.enroll_student(integer,integer,character) line 17 at RAISE SQL state: P0001	

Q7. Procedure with OUT Parameters

Create academics.dept_summary(p_dept_id INTEGER, OUT student_count INTEGER, OUT avg_cgpa NUMERIC) that fills the OUT parameters with the count of students and average CGPA for that department. Test with CALL.

```
445 CREATE OR REPLACE PROCEDURE academics.dept_summary(  
446     p_dept_id INTEGER,  
447     OUT student_count INTEGER,  
448     OUT avg_cgpa NUMERIC  
449 )  
450 LANGUAGE plpgsql  
451 AS $$  
452 BEGIN  
453  
454     SELECT  
455         COUNT(*)::INTEGER,  
456         AVG(cgpa)  
457     INTO  
458         student_count,  
459         avg_cgpa  
460     FROM academics.students  
461     WHERE dept_id = p_dept_id;  
462  
463 END;  
464 $$;
```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 59 msec.

Test:

```
515 CALL academics.dept_summary(1, NULL, NULL);  
516  
517
```

Data Output Messages Notifications

SQL

Showing rows: 1 to 1 Page No: 1 of 1

	student_count integer	avg_cgpa numeric
1	5	8.5400000000000000

Q8. Procedure — Transfer Student

Create academics.transfer_student(p_student_id INTEGER, p_new_dept_id INTEGER) that:

- Validates both student and new department exist
- Updates students.dept_id
- Raises NOTICE on success
- Raises EXCEPTION on any validation failure

Test a successful transfer and a failure case.

```
519 CREATE OR REPLACE PROCEDURE academics.transfer_student(  
520     p_student_id INTEGER,  
521     p_new_dept_id INTEGER  
522 )  
523 LANGUAGE plpgsql  
524 AS $$  
525 BEGIN  
526     IF NOT EXISTS (  
527         SELECT 1  
528         FROM academics.students  
529         WHERE student_id = p_student_id  
530     ) THEN  
531         RAISE EXCEPTION 'Student ID % does not exist',  
532             p_student_id;  
533     END IF;  
534     IF NOT EXISTS (  
535         SELECT 1  
536         FROM academics.departments  
537         WHERE dept_id = p_new_dept_id  
538     ) THEN  
539         RAISE EXCEPTION 'Department ID % does not exist',  
540             p_new_dept_id;  
541     END IF;  
542     UPDATE academics.students  
543     SET dept_id = p_new_dept_id  
544     WHERE student_id = p_student_id;  
545     RAISE NOTICE 'Student % transferred successfully to department %',  
546         p_student_id,  
547         p_new_dept_id;  
548 END;  
549 $$;
```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 96 msec.

Check before transfer

```
551 SELECT student_id, student_name, dept_id  
552 FROM academics.students  
553 WHERE student_id = 1;
```

Data Output Messages Notifications

	student_id [PK] integer	student_name character varying (100)	dept_id integer
1	1	Sunakshi	1

Successful transfer

```
556 CALL academics.transfer_student(1, 2);
```

557
558

Data Output Messages Notifications

NOTICE: Student 1 transferred successfully to department 2
CALL

Query returned successfully in 90 msec.

Check after transfer

```
558 SELECT student_id, student_name, dept_id  
559 FROM academics.students  
560 WHERE student_id = 1;
```

Data Output	Messages	Notifications
student_id	student_name	dept_id
[PK] integer	character varying (100)	integer
1	Sunakshi	2

Failure — invalid student

```
562 CALL academics.transfer_student(9999, 2);
```

563

Data Output Messages Notifications

NOTICE: Student 9999 transferred successfully to department 2
CALL

Query returned successfully in 76 msec.

Failure — invalid department

```
564 CALL academics.transfer_student(1, 9999);
```

Data Output Messages Notifications

ERROR: Department ID 9999 does not exist
CONTEXT: PL/pgSQL function academics.transfer_student(integer,integer) line 16 at RAISE
SQL state: P0001

Q9. Function Using Control Structures

Create `academics.letter_from_cgpa(p_cgpa NUMERIC)` that returns a letter grade using IF/ELSIF:

$\geq 9 \rightarrow 'A+', \geq 8 \rightarrow 'A', \geq 7 \rightarrow 'B+', \geq 6 \rightarrow 'B', \geq 5 \rightarrow 'C',$ else 'F'. Handle NULL.

```
568 CREATE OR REPLACE FUNCTION academics.letter_from_cgpa(  
569     p_cgpa NUMERIC  
570 )  
571 RETURNS TEXT  
572 LANGUAGE plpgsql  
573 AS $$  
574 BEGIN  
575  
576     IF p_cgpa IS NULL THEN  
577         RETURN NULL;  
578     ELSIF p_cgpa >= 9 THEN  
579         RETURN 'A+';  
580     ELSIF p_cgpa >= 8 THEN  
581         RETURN 'A';  
582     ELSIF p_cgpa >= 7 THEN  
583         RETURN 'B+';  
584     ELSIF p_cgpa >= 6 THEN  
585         RETURN 'B';  
586     ELSIF p_cgpa >= 5 THEN  
587         RETURN 'C';  
588     ELSE  
589         RETURN 'F';  
590     END IF;  
591  
592 END;  
593 $$;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 54 msec.

Tests

1. A+

```
596 SELECT academics.letter_from_cgpa(9.5);
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	letter_from_cgpa
1	A+

2. A

```
600 SELECT academics.letter_from_cgpa(8.5);
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	letter_from_cgpa text
1	A

3. B+

600

SELECT academics.letter_from_cgpa(7.5);

Data Output

Messages

Notifications



Showing rows: 1 to 1

Page No: 1 of 1

	letter_from_cgpa text
1	B+

4. F

```
600 SELECT academics.letter_from_cgpa(4.5);
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	letter_from_cgpa text
1	F

5. NULL

600

SELECT academics.letter_from_cgpa(NULL);

Data Output

Messages

Notifications



Showing rows: 1 to 1

Page No: 1 of 1

	letter_from_cgpa text
1	[null]

Q10. Set-Returning Function with Join Logic

Create `academics.student_course_report(p_student_id INTEGER)` that RETURNS TABLE(course_name VARCHAR, grade CHAR(2), points NUMERIC). It should return all courses the student is enrolled in along with grade and grade points (reuse `grade_to_points`). Test it.

```
604 CREATE OR REPLACE FUNCTION academics.student_course_report(  
605     p_student_id INTEGER  
606 )  
607 RETURNS TABLE(  
608     course_name VARCHAR,  
609     grade CHAR(2),  
610     points NUMERIC  
611 )  
612 LANGUAGE plpgsql  
613 AS $$  
614 BEGIN  
615  
616     RETURN QUERY  
617     SELECT  
618         c.course_name,  
619         e.grade,  
620         academics.grade_to_points(e.grade)  
621     FROM academics.enrollments e  
622     JOIN academics.courses c  
623         ON e.course_id = c.course_id  
624     WHERE e.student_id = p_student_id;  
625  
626 END;  
627 $$;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 78 msec.

Test

```
630 SELECT *  
631 FROM academics.student_course_report(1);  
632  
633
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	course_name character varying	grade character	points numeric
1	Database Management System...	A	9
2	Computer Networks	B+	8
3	Operating Systems	A	9

Q11. Procedure — Bulk Grade Update

Create a procedure `academics.bulk_update_grade(p_course_id INTEGER, p_old_grade CHAR(2), p_new_grade CHAR(2))` that updates all enrollments for that course from `old_grade` to `new_grade`. Raise NOTICE with the number of rows updated. Test it.

```
634 CREATE OR REPLACE PROCEDURE academics.bulk_update_grade(  
635     p_course_id INTEGER,  
636     p_old_grade CHAR(2),  
637     p_new_grade CHAR(2)  
638 )  
639 LANGUAGE plpgsql  
640 AS $$  
641 DECLARE  
642     v_rows_updated INTEGER;  
643 BEGIN  
644     UPDATE academics.enrollments  
645     SET grade = p_new_grade  
646     WHERE course_id = p_course_id  
647         AND grade = p_old_grade;  
648  
649     GET DIAGNOSTICS v_rows_updated = ROW_COUNT;  
650  
651     RAISE NOTICE '% rows updated',  
652         v_rows_updated;  
653 END;  
654 $$;
```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 112 msec.

Verify

```
661 SELECT *  
662 FROM academics.enrollments  
663 WHERE course_id = 2;
```

Data Output Messages Notifications

	student_id [PK] integer	course_id [PK] integer	enroll_date date	grade character (2)
1	2	2	2026-08-01	A+
2	3	2	2026-08-04	B+
3	1	2	2026-08-01	A+
4	4	2	2026-08-04	A+

Q12. Drop & Document

- (a) List the functions and procedures you created (query pg_proc or just note the names).
- (b) DROP any experimental ones you no longer need.
- (c) In 2-3 sentences explain when you would choose a Function over a Procedure (and vice versa).

(a) List Functions and Procedures

Functions

```
683
684 SELECT
685     n.nspname AS schema_name,
686     p.proname AS function_name
687 FROM pg_proc p
688 JOIN pg_namespace n
689     ON p.pronamespace = n.oid
690 WHERE n.nspname = 'academics'
691     AND p.prokind = 'f'
692 ORDER BY p.proname;
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1

	schema_name name	function_name name
1	academics	calc_gpa
2	academics	cgpa_category
3	academics	get_dept_students
4	academics	grade_to_points
5	academics	letter_from_cgpa
6	academics	student_course_rep...

Procedures

```
695 SELECT
696     n.nspname AS schema_name,
697     p.proname AS procedure_name
698 FROM pg_proc p
699 JOIN pg_namespace n
700     ON p.pronamespace = n.oid
701 WHERE n.nspname = 'academics'
702     AND p.prokind = 'p'
703 ORDER BY p.proname;
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1

	schema_name name	procedure_name name
1	academics	bulk_update_grade
2	academics	dept_summary
3	academics	enroll_student
4	academics	transfer_student

(b) DROP Experimental Ones

Not required.

(c) Function vs Procedure

A Function is used when we need to calculate or return a value and it can be called using SELECT. A Procedure is preferred when we want to perform an action such as inserting, updating, transferring or modifying data and we call it using CALL. Procedures are useful for transaction-oriented operations and can use OUT parameters and transaction control features.